



**Universidad
de Santander**
UDES

ACREDITACIÓN
INTERNACIONAL
EQUAA
Education Quality Accreditation Agency

Patrones de Diseño de Software

Unidad 10: Métricas de Calidad y SonarQube

Post-Contenido 2

Ingeniería de Sistemas

2026

Objetivo de la Actividad

El estudiante configura un Quality Gate personalizado en SonarQube, corrige al menos tres Code Smells y un Bug identificados en el análisis del Post-Contenido 1, ejecuta un segundo análisis que confirme la mejora de las métricas, e integra el análisis en el pipeline de GitHub Actions para automatizar la inspección en cada push.

Prerrequisitos

- Haber completado Post-Contenido 1: SonarQube corriendo en Docker con el primer análisis ejecutado
- El repositorio del Post-Contenido 1 con los hallazgos documentados
- Cuenta en GitHub con el repositorio del Post-Contenido 1 subido
- JDK 21, Maven 3.9+ y Docker Desktop en ejecución

Descripción del Laboratorio

El laboratorio cierra el ciclo de inspección de calidad iniciado en el Post-Contenido 1: configurar un Quality Gate que establezca condiciones mínimas de aceptación, corregir los problemas más relevantes detectados por SonarQube, re-ejecutar el análisis para confirmar la mejora, y automatizar el proceso mediante GitHub Actions.

Paso 1: Configurar un Quality Gate Personalizado

Desde el dashboard de SonarQube (<http://localhost:9000>), crear un Quality Gate personalizado:

- Navegar a Quality Gates → Create → Nombre: "Estándar Universidad"
- Agregar condición: Bugs is greater than 0 → bloquear si hay al menos 1 bug
- Agregar condición: Coverage is less than 60% → cobertura mínima del 60%
- Agregar condición: Code Smells is greater than 5 → máximo 5 code smells permitidos
- Agregar condición: Duplicated Lines (%) is greater than 5% → máximo 5% de duplicación

Asignar el Quality Gate al proyecto: en el dashboard del proyecto → Project Settings → Quality Gate → seleccionar "Estándar Universidad".

Checkpoint de Verificación

- El Quality Gate "Estándar Universidad" aparece en Quality Gates → All con las 4 condiciones configuradas.
- El proyecto "Productos Service" tiene asignado el Quality Gate personalizado.
- Al ejecutar mvn sonar:sonar, el dashboard muestra el estado del Quality Gate (probable "Failed" inicialmente).

Paso 2: Corregir el Bug Crítico — orElse(null)

El bug más importante identificado en el Post-Contenido 1 es el retorno de null en el método buscar(). El retorno de null es una fuente frecuente de NullPointerException en quien llama al método. La corrección consiste en lanzar una excepción descriptiva:

```
// ANTES (Bug: retorna null, causa NullPointerException en el llamador)
public Producto buscar(Long id) {
    return repo.findById(id).orElse(null);
}

// DESPUÉS (corrección: lanzar excepción con mensaje descriptivo)
public Producto buscar(Long id) {
    return repo.findById(id)
        .orElseThrow(() ->
            new NoSuchElementException("Producto no encontrado: " +
id));
}
```

Checkpoint de Verificación

- El método buscar() lanza NoSuchElementException en lugar de retornar null.
- Se agrega una prueba unitaria que verifica que buscar() lanza la excepción para un id inexistente.

Paso 3: Corregir Code Smells de Mayor Impacto

Corregir al menos tres de los siguientes Code Smells identificados en el análisis:

3.1 Inyección por Constructor (reemplazar @Autowired en campo)

```
// ANTES (Code Smell: @Autowired en campo, dificulta pruebas)
@Autowired
private ProductoRepository repo;

// DESPUÉS (inyección por constructor — recomendada por Spring)
```

```
private final ProductoRepository productoRepository;

public ProductoService(ProductoRepository productoRepository) {
    this.productoRepository = productoRepository;
}
```

3.2 Reemplazar n.equals("") por isBlank()

```
// ANTES (Code Smell: comparación de cadena vacía con equals)
if (n == null || n.equals("")) {
    throw new IllegalArgumentException("nombre requerido");
}

// DESPUÉS (uso correcto de isBlank() - cubre null, "", " ")
if (nombre == null || nombre.isBlank()) {
    throw new IllegalArgumentException("El nombre no puede estar vacío");
}
```

3.3 Extraer Método para Reducir Complejidad Ciclomática

El método procesarProducto() tiene CC alta por concentrar múltiples validaciones. Se refactoriza extrayendo la validación a un método privado:

```
// Método principal - reducido a orquestación
public Producto procesarProducto(String nombre, Double precio, Integer stock) {
    validarDatos(nombre, precio, stock);
    Producto producto = new Producto();
    producto.setNombre(nombre.strip());
    producto.setPrecio(precio);
    producto.setStock(stock);
    return productoRepository.save(producto);
}

// Método extraído - validación separada (CC reducida)
private void validarDatos(String nombre, Double precio, Integer stock) {
    if (nombre == null || nombre.isBlank())
        throw new IllegalArgumentException("El nombre no puede estar vacío");
    if (precio == null || precio <= 0)
        throw new IllegalArgumentException("El precio debe ser mayor a cero");
    if (precio > 999999)
        throw new IllegalArgumentException("El precio excede el máximo permitido");
    if (stock == null || stock < 0)

```

```
throw new IllegalArgumentException("El stock no puede ser  
negativo");  
}
```

Checkpoint de Verificación

- La inyección por constructor reemplaza @Autowired en campo en ProductoService.
- El método procesarProducto() usa nombre.isBlank() en lugar de nombre.equals("").
- La complejidad ciclomática del método principal se redujo extrayendo validarDatos().
- mvn clean verify ejecuta sin errores tras las refactorizaciones.

Paso 4: Re-ejecutar el Análisis y Verificar la Mejora

Tras las correcciones, ejecutar el segundo análisis de SonarQube:

```
# Ejecutar pruebas + JaCoCo + análisis SonarQube  
mvn clean verify sonar:sonar -Dsonar.token=TU_TOKEN
```

Verificar en el dashboard que:

- La cantidad de Bugs se redujo (el retorno null fue corregido).
- La cantidad de Code Smells disminuyó (al menos 3 corregidos).
- El Quality Gate "Estándar Universidad" ahora pasa o indica exactamente qué condición falta.
- La cobertura mejoró si se agregaron pruebas junto a las correcciones.

Checkpoint de Verificación

- El segundo análisis muestra reducción en el número de Bugs respecto al análisis inicial.
- Al menos 3 Code Smells menos que en el análisis inicial del Post-Contenido 1.
- El dashboard de SonarQube muestra el historial de dos análisis con mejora visible.
- Captura del segundo dashboard incluida en el README comparando con el estado inicial.

Paso 5: Integrar SonarQube en GitHub Actions

Para automatizar el análisis en cada push, agregar el paso de SonarQube al workflow de GitHub Actions. Se requiere configurar el token como secreto del repositorio (Settings → Secrets and variables → Actions → New repository secret → SONAR_TOKEN):

```
# .github/workflows/ci.yml – agregar tras el paso de verify
name: CI con SonarQube

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Necesario para análisis diferencial de
            SonarQube

      - uses: actions/setup-java@v4
        with:
          java-version: "21"
          distribution: "temurin"
          cache: maven

      - name: Compilar, probar y analizar
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
        run: |
          mvn -B clean verify sonar:sonar \
            -Dsonar.host.url=https://sonarcloud.io \
            -Dsonar.token=${ env.SONAR_TOKEN } \
            --no-transfer-progress
```

Nota: Para GitHub Actions con servidor público, se recomienda usar SonarQube Cloud (sonarcloud.io) en lugar del servidor local, ya que el servidor local en Docker no es accesible desde los runners de GitHub. SonarQube Cloud es gratuito para proyectos públicos. Alternativamente, comentar el paso de SonarQube en el workflow y documentar que el análisis se ejecuta localmente.

Checkpoint de Verificación

- El workflow de GitHub Actions ejecuta mvn clean verify correctamente en el runner de GitHub.
- El README incluye el badge del workflow y la tabla comparativa antes/después del análisis.
- El repositorio tiene al menos 3 commits: análisis inicial documentado, correcciones aplicadas, segunda ejecución.
- El README muestra capturas comparativas: dashboard antes de las correcciones y después.

Rúbrica de Evaluación

Criterio	Excelente (90–100%)	Bueno (70–89%)	Aceptable (50–69%)	Insuficiente (0–49%)
Implementación (40%)	Proyecto configurado correctamente con SonarQube y JaCoCo; análisis ejecuta sin errores; todas las clases requeridas presentes y funcionales.	La mayor parte de la configuración es correcta; existen errores menores que no impiden la ejecución del análisis.	Configuración incompleta o con errores que requieren correcciones; el análisis se ejecuta parcialmente.	No se ejecuta el análisis o la configuración es fundamentalmente incorrecta; no evidencia comprensión del laboratorio.
Funcionalidad (25%)	El reporte de SonarQube muestra hallazgos correctamente; JaCoCo reporta cobertura; el dashboard refleja el estado real del proyecto.	El reporte muestra la mayoría de hallazgos esperados; cobertura visible aunque con alguna configuración incompleta.	El reporte muestra resultados parciales; la cobertura no aparece correctamente integrada con SonarQube.	El dashboard de SonarQube no muestra hallazgos o muestra datos incorrectos; no hay evidencia de integración con JaCoCo.
Documentación (20%)	README con instrucciones claras de ejecución, capturas del dashboard de SonarQube y descripción de	README presente con capturas básicas del dashboard; commits con mensajes adecuados.	README mínimo sin capturas del dashboard; algunos commits sin mensajes descriptivos.	Sin README o documentación del análisis; repositorio sin capturas de evidencia.



Criterio	Excelente (90–100%)	Bueno (70– 89%)	Aceptable (50–69%)	Insuficiente (0–49%)
	los hallazgos encontrados; commits descriptivos.			
Estilo y convenciones (15%)	Código organizado en la estructura de paquetes definida; configuración sonar-project.properties correctamente comentada.	Estructura mayormente correcta con inconsistencias menores de organización.	Estructura parcialmente correcta; algunos archivos en ubicaciones incorrectas.	Estructura de paquetes no respeta las convenciones; configuración sin comentarios.

Entregables

Elemento	Especificación
Plataforma	GitHub (repositorio público)
Nombre del repositorio	apellido-post2-u10
Estructura requerida	README.md + src/ con código fuente + capturas del dashboard de SonarQube
Commits mínimos	Mínimo 3 commits con mensajes descriptivos que reflejen el progreso del laboratorio
Evidencia de análisis	Capturas de pantalla del dashboard de SonarQube incluidas en el README